

JAVASCRIPT VARIABLES INTERVIEW QUESTIONS

1. What is a variable in JavaScript?

A variable is a container that holds data.

You can think of it as a label for a specific spot in memory where information is stored.

Variables are essential because they allow you to store, retrieve, and manipulate data throughout your program.

In the code snippet below, me is a variable:

```
const me = {  
  name: "Ndeye Fatou Diop",  
  age: 30,  
}  
  
console.log(`Your name is ${me.name} and you are ${me.age}.`)
```

2. What is the difference between declaration, initialization, and re-assignment?

Declaring a variable means specifying its type (e.g., let) and name.

Initializing a variable means giving it a value.

Reassigning a variable means changing its value. Only variables declared with `let` or `var` can be assigned a new value.

```
let name; // This is a declaration
```

```
name = "Ndeye Fatou Diop" // This is an initialization
```

```
name = "Fatou" // This is a re-assignment
```

3. How do you declare a variable in JavaScript?

There are **3** main ways to declare variables in JavaScript:

Option 1: Using `let`

```
let age;
```

Option 2: Using `const`

// IMPORTANT: We cannot do only `const name;` we must initialize the variable

```
const name = "Ndeye Fatou Diop";
```

Option 3 (old way, not recommended): Using `var`

```
var name;
```

4. What is strict mode in JavaScript?

Strict mode is a way to opt into a restricted variant of JavaScript.

When you enable strict mode, the JavaScript engine will enforce stricter parsing and error handling rules.

For example,

You can't assign to undeclared variables (which would create a global variable).

- You can't assign to a non-writable global (e.g., `undefined = 5`).
- You can't duplicate parameter names (e.g., `function sum(a, a, b) {...}`).

To enable strict mode, add `'use strict';` at the beginning of your script or function.

```
'use strict';
```

```
function sum(a, a, b) {  
    return a + b;  
}
```

```
console.log("Sum: ", sum(1, 3, 2)) // SyntaxError: Duplicate parameter name  
not allowed in this context
```

5. What are some best practices for naming variables in JavaScript?

When naming variables in JavaScript, follow these best practices:

1. **Avoid reserved words:** You cannot use JavaScript reserved words (like `class`, `var`, `function`, etc.) as variable names. These are keywords used by the language itself. See the full list [here](#).
2. **Use camelCase for most variables:** The convention is to use camelCase for naming variables. This means the first word is lowercase, and each subsequent word starts with a capital letter. Constants, on the other hand, should be written in all uppercase with underscores separating words.
3. **Be descriptive:** Variable names should describe what the variable is used for. This makes your code more readable. For tiny scopes (like loop headers), short names like `i` are Ok; otherwise, avoid vague names like `x` or `data`.

// ❌ Bad

let try; // try is a reserved word

let place_of_birth; // not camelCase or uppercase

const x = 100; // not descriptive

// ✅ Good

let shouldRetry;

let placeOfBirth;

const TIMEOUT_MS = 100;

6. What does "scope" mean in the context of JavaScript variables

In JavaScript, **scope** refers to the area of the code where a variable can be accessed or used.

If a variable or expression is not in the current scope, it will not be available for use.

Think of JavaScript scope like a house 🏠 with different rooms.

Each room has its own set of stuff that can only be accessed by the person in that room.

7. What happens if you try to redeclare a let or const variable?

You will get an error if you try to redeclare a let, const variable **within the same scope**.

// ❌ Bad: We are redeclaring `fullName` within the same scope

if(myCondition) {

```
// We are inside a block scope
let fullName = "Ndeye Fatou";
// ...
let fullName; // SyntaxError: Identifier 'fullName' has already been
declared
}
// ✅ This works
if(myCondition) {
  let fullName = "Ndeye Fatou";
  // ...
}
let fullName; // This is Ok since `fullName` is declared outside of the block
scope
```

8. What is "hoisting" in JavaScript?

Hoisting is a behavior where variable and function declarations are moved to the top of their containing scope during the compilation phase before the code is executed.

This means that you can reference some variables or functions before they are declared in the code.

- **Function declarations are fully hoisted**, meaning you can call a function before its actual declaration in the code.

```
sayHello(); // This works, even though the function is declared later in the
code
```

```
function sayHello() {
```

```
console.log("Hello!");  
}
```

- **Variables declared with var** are hoisted and initialized as undefined. This means you can access the variable before it's declared in the code.

```
console.log(name); // This will log `undefined`
```

```
var name = "Ndeye Fatou"
```

```
console.log(name); // This will log `Ndeye Fatou`
```

- **Variables declared with let and const** are hoisted but not initialized. This means you can't access the variable before it's declared in the code.

```
console.log(name); // This will throw an error `ReferenceError: Cannot  
access 'name' before initialization`
```

```
let name = "Ndeye Fatou"
```

```
console.log(name); // This will log `Ndeye Fatou`
```

9. What are the differences between var, let, and const?

1. Smallest Scope:

- var: Function-scoped
- let: Block-scoped
- const: Block-scoped

2. Re-declaration:

- var: Allowed within the same scope
- let: Not allowed within the same scope
- const: Not allowed within the same scope

3. Re-assignment:

- var: Allowed
- let: Allowed
- const: Not allowed (must be initialized when declared)

4. Hoisting:

- var: Hoisted but initialized as undefined
- let: Hoisted but not initialized
- const: Hoisted but not initialized

5. Best Practice:

- var: DO NOT USE (can cause confusing bugs)
- let: Preferred for variables that change
- const: Preferred for variables that don't change

10. What happens if you try to use a variable before it is declared?

It depends on the variable type and the mode.

- If you try to use let and const before declaring them, you will get a ReferenceError.
- If you try to use var, you will get undefined.

11. How do you determine the type of a variable in JavaScript?

This is a trick question 😊.

In JavaScript, variables don't have types—values do.

When you use `typeof myVariable`, you're checking the type of value the variable holds.